

```
<xlrd.book.Book object at 0x000000002F760B8>
```



Note: To qualify a file name via the *open_workbook* API in Windows, I had to escape the “\” backslash in the path with another, to indicate that I am not using any special sequence.

Once a book is available, we can open a sheet of the same.

```
>>> testsheet=testbook.sheet_by_index (0)
>>> testsheet
<xlrd.sheet.Sheet object at 0x000000002F73A90>
```

From the sheet, we print the following values. You can see the values and types associated with the values.

```
>>> testsheet.cell (1,0)
xldate:42036.0
>>> testsheet.cell (1,1)
text:u 'ProjectA'
>>> testsheet.cell (1,2)
text:u 'Jackie Chain'
>>> testsheet.cell (1,3)
number:10.0
```



Note: Indexes of both rows and columns start with 0 (zero), which is good for us as we naturally escape the header row.

The date, as can be seen above, is stored as a number, which *xlrd* shows as an *xldate* type. Let us do some more processing with the same. To work with Python dates, we would need the date and datetime classes from the datetime module.

```
>>> from datetime import date, datetime
>>> date.today()
date.date(2015, 2, 1)
```

We can see that the date ‘1 Feb 2015’ is represented in Python with the year, month and date components.

To work with the *xldate* type, let us import and use the *xldate_as_tuple* interface.

```
>>> from xlrd import xldate_as_tuple
>>> testdate=xldate_as_tuple (testsheet.
    cell(1,0).value, testbook.datemode)
>>> testdate
(2015, 2, 1, 0, 0, 0)
```

We can see that the same date is stored as a datetime internally and has hours, minutes and seconds components too.

To extract only the date part and ignore the last three, we can do a sub-tuple selection and initialise the date with the same.

```
>>> testdate
(2015, 2, 1, 0, 0, 0)
>>> date (*testdate[:3])
datetime.date (2015, 2, 1)
```

Now, simple comparisons can be done between Python date classes in the following way.

```
>>> date.today()==date(*testdate[:3])
True
```



Note: In the above segment, we make a slice of *testdate* starting from 0 to 2 and <3, with only the first three values from *testdate* to get date components of the year, month and day. We use the * operator to unpack and list the values to be used to construct a date object.

Designing the program

So, having set those items up, let us come back to the original problem. We need to have a program that summarises the efforts by the customer's name and prints it. Let us assume that the data now looks like what's shown in Figure 3 in a file *c:\test.xlsx*:

Given that all reports will be in a particular directory, we will need a way to get a list of the report files so that we can process them in our program. Python provides us with the *glob* object which helps to do that.

```
>>> import glob
>>> glob.glob ("C:\*.xlsx")
['C:\\test.xlsx']
```

Here's the sequence for the program.

1. Create a dictionary object to store *nHours* for each customer name.
2. Using the *glob* object, fetch the list of *xlsx* report files.
3. For each report file:
 - a. Cycle through each row of sheet (0)
 - b. For each row with valid entry...
 - i. Check if the date is present and is valid
 - ii. Check if the date is present and the date is within the last seven days
 1. Fetch the hours spent
 2. Fetch the customer's name
 3. Customer hours + hours spent
4. Print the final output from the map
 - a. Print the customer's name and the total hours spent on the customer

The following is the complete program:

```
""" summarize effort values from xlsx reports and print the
same.
```